

Bare-Metal + FreeRTOS + Networking on Raspberry Pi 1 / 3 / 400

A from-scratch, hardware-confirmed embedded platform with an ESP32-S3 instrument bridge

A from-scratch, hardware-confirmed embedded platform. No Linux, no vendor BSP, no pre-built driver stack. Every layer — boot, scheduler, USB host, Ethernet MAC, TCP/IP glue, display, and peripheral control — was brought up on real silicon and verified on the wire and on the screen, across three different Raspberry Pi SoC generations and a companion ESP32-S3 that turns the rig into a logic analyzer and oscilloscope.

Repositories: - Raspberry Pi: github.com/cecep1105/RPi-FreeRTOS-RealSilicon-Demo - ESP32-S3: github.com/cecep1105/ESP32-FreeRTOS_Demo

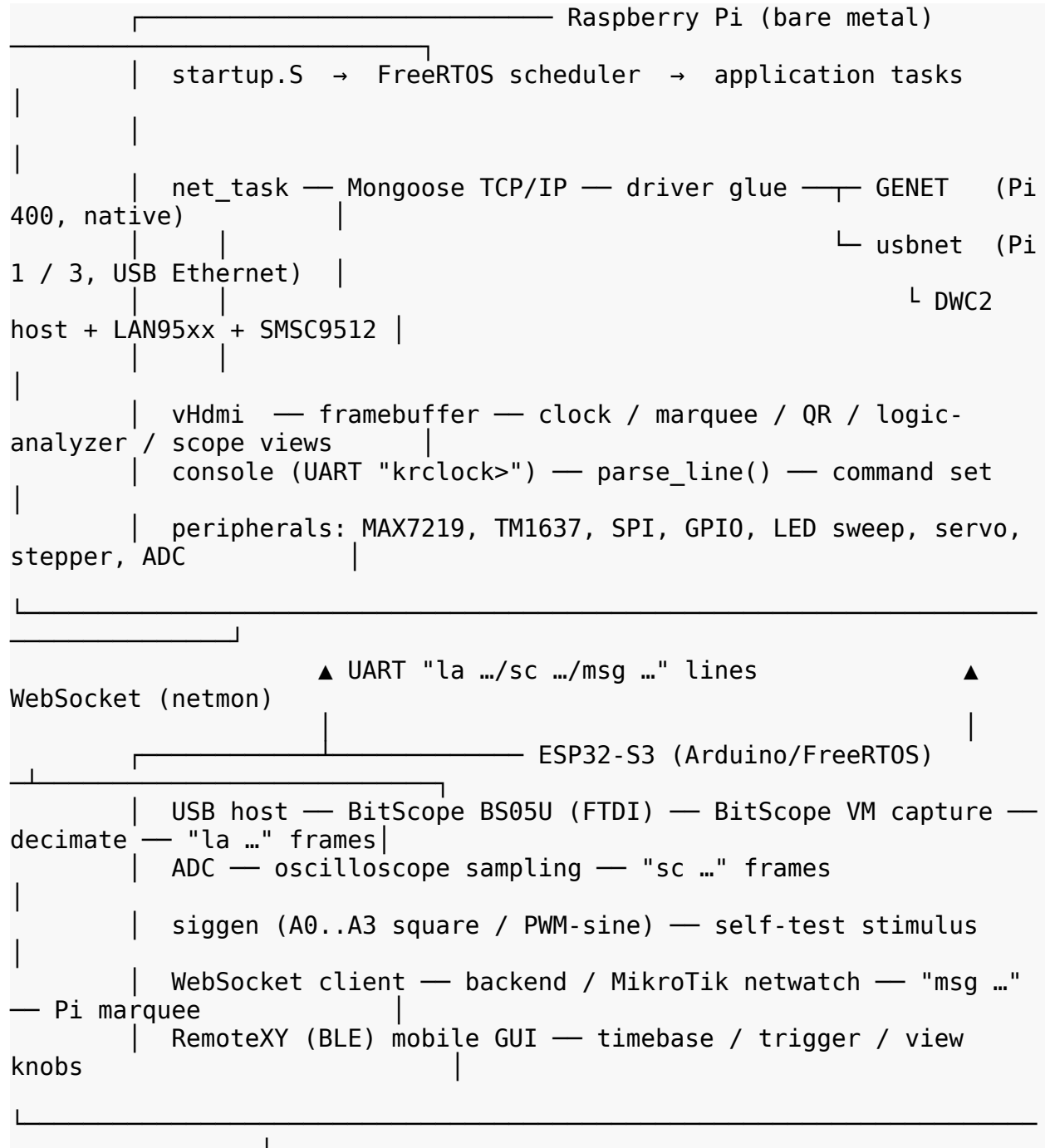
1. What this project is

The goal was a *single coherent design* that runs the same feature set across deliberately different hardware, proving the drivers are real and portable rather than copied from an OS. The same application — a networked clock/dashboard with a scrolling marquee, QR rendering, an HDMI instrument view, and a UART command console — runs on:

Target	SoC	Core	ISA	Ethernet	Notable constraint
Pi 1 (Model B / B+)	BCM2835	ARM1176JZF-S	ARMv6 (32-bit)	USB (SMSC9512)	No MMU; strongly-ordered memory; no hardware unaligned access
Pi 3	BCM2837	Cortex-A53	ARMv8 (AArch64)	USB (LAN9514)	DMA via uncached alias
Pi 400	BCM2711	Cortex-A72	ARMv8 (AArch64)	Native GENET	Pi 4 family; on-die gigabit MAC
ESP32-S3	ESP32-S3	Xtensa LX7	—	Wi-Fi / BLE	Arduino-framework companion; USB host

The two AArch64 boards share one codebase; the ARMv6 board shares the *same driver sources* with a small set of architecture guards. Where the silicon genuinely differs — DMA coherence, memory ordering, unaligned access — the difference is isolated behind a handful of `#if defined(__aarch64__)` seams, not forked files.

2. System architecture



Boot and scheduler

Each Pi boots from its own `startup.S` into a minimal C runtime, sets up the stack and BSS, and starts the FreeRTOS scheduler. The Pi 1 links **newlib** (`-specs=nosys.specs`) with a soft-float ARMv6 target; the AArch64 boards build `-nostdlib` with hand-written libc shims (`kstring.c`). Tick rate is 1 kHz. Networking is brought up *before* the scheduler starts (USB enumeration is long and synchronous), then handed to a polling `net_task`.

Networking stack

Mongoose provides the TCP/IP stack and HTTP/WebSocket/DNS/SNTP clients. It was integrated **unmodified** — a deliberate constraint, so all three targets share a byte-identical `mongoose.c`. The driver glue presents a uniform `genet_net_*` interface; on the USB boards that interface forwards to `usbnet_*`, on Pi 400 it drives GENET directly. Services confirmed on hardware: DHCP lease (with static fallback after a timeout), NTP time sync, an HTTP dashboard behind HTTP Basic Auth, and a WebSocket client that relays push notifications to the marquee.

USB Ethernet from scratch (Pi 1 and Pi 3)

Neither USB board has a native MAC. The full chain was written from the register level up: - **DWC2 host controller** — channel setup, SETUP/IN/OUT transfers, toggle management, NAK/halt handling, the root port and a hub. - **LAN9514 (Pi 3) / LAN9512 (Pi 1) hub + SMSC9512 NIC** — enumeration, MAC address via the VideoCore mailbox, PHY bring-up, MAC control, bulk IN/OUT for frames. - **Glue into Mongoose** — `usbnet_tx` / `usbnet_rx` move Ethernet frames between the bulk endpoints and Mongoose's buffers.

This is the part where the hardware fought back, and where most of the engineering went.

3. The hard problems (and how each was nailed)

Three of these are ARMv6-specific. They are the reason Pi 1 was the long pole, and they are the most reusable lessons in the project — every one is a silent, data-corrupting failure that compiles cleanly and only shows up on real hardware.

3.1 DMA cache coherence on ARMv6 — the L2 alias

On Pi 1 the CPU reaches SDRAM through the VideoCore L2 cache, but the USB controller's DMA does not. Using the uncached `0xC0000000` bus alias (correct on Pi 3/400, where DMA buffers are mapped non-cacheable) produced **intermittent stale reads** on Pi 1: SETUP packets sent stale, descriptor reads returning the wrong structure, port status flickering between 0 and 5. The fix is a per-architecture bus-address macro:

```
#if defined(__aarch64__)
# define USB_BUS_ADDR(p) (((uint32_t)(uintptr_t)(p)) | 0xC0000000u)
// Pi 3/400: uncached
#else
# define USB_BUS_ADDR(p) (((uint32_t)(uintptr_t)(p)) | 0x40000000u)
```

```
// Pi 1: L2-COHERENT
#endif
```

plus a data-sync barrier after channel halt (USB_DSB(), itself arch-guarded: dsb sy on AArch64, the CP15 c7, c10, 4 barrier on ARMv6). With the L2-coherent alias the CPU and DMA agree, and enumeration becomes reliable.

3.2 The DHCP xid bug — ARMv6 unaligned access (the headline)

After everything else worked on Pi 1 — USB up, link up, Mongoose running, NTP succeeding, ARP answering — **DHCP would never complete**. Every OFFER arrived and was visibly dropped. The investigation is worth recording because the symptom pointed everywhere except the real cause:

1. The OFFER arrived intact at the driver (confirmed with a raw hex dump: correct Ethernet/IP/UDP headers, correct lengths, valid checksums).
2. Mongoose dropped it before the DHCP handler ran. Bracketing logs showed the transaction ID check failing: the wire xid was EB 91 B8 27 but ours was EB 91 10 CD.
3. That value is not random — it is `mac[2] mac[3] mac[0] mac[1]`, the exact result of an **ARMv6 legacy “rotate” unaligned 32-bit load**: a word LDR from a 2-byte-aligned address reads the aligned container and rotates right by $8 \times (\text{addr} \& 3)$ bits.

Root cause: Mongoose builds the xid with `memcpy(&dhcp.xid, ifp->mac + 2, 4)`. Because `ifp->mac` lives in an 8-byte-aligned union, `mac + 2` is unaligned, and **newlib’s ARMv6 memcpy (and GCC’s inlined small copies) assume hardware unaligned access** — which the Pi 1 does not have with the MMU off. Every other protocol field survived because Mongoose reads headers through byte-wise `MG_LOAD_BE*` macros; the xid was the one field built from a raw `memcpy`.

The fix is two complementary parts, both Pi 1-only: - **aligned_mem.c** — a strong `memcpy/memmove` override that does word copies *only* when both pointers are word-aligned and byte-copies otherwise. This catches real calls, including newlib’s. - **-mno-unaligned-access** — stops GCC emitting *inlined* unaligned word loads (which no symbol override can intercept). It converts them into real `memcpy` calls or byte loads.

Neither alone is sufficient; together they make every unaligned access on Pi 1 safe. Critically, **Mongoose stays stock** — the fix lives entirely in the libc layer and a build flag, so all three targets keep an identical `mongoose.c`. (A self-test that does `memcpy(&x, mac+2, 4)` and prints the result — expecting `0xCD1091EB`, the bad value being `0x27B891EB` — was the decisive instrument that confirmed the flag had actually taken.)

3.3 The broadcast filter — SMSC MAC_CR

The SMSC9512’s MAC control register defaults to a perfect-address filter that **drops all broadcast frames** — which includes ARP who-has and broadcast DHCP. Without the broadcast bit the host can’t resolve the Pi and DHCP can stall. One line fixes it:

```
smsc_write_reg(SMSC_MAC_CR, SMSC_MAC_CR_TXEN | SMSC_MAC_CR_RXEN |
SMSC_MAC_CR_BCAST);
```

3.4 Hub bring-up robustness

The LAN9514's internal Ethernet bridge can assert its downstream connect *later* than a single port-status read. A fixed delay plus one scan was replaced with **honoring the hub's own power-on-to-power-good time** (bPwrOn2PwrGood, floored at 100 ms) and then **polling** for the first connected port for up to ~2 s. This removed a class of cold-boot flakiness.

3.5 One driver, three architectures

The payoff of the arch guards: the *same* `usb_dwc2.c/.h` compile and run on ARMv6 and AArch64. The only differences are `USB_BUS_ADDR`, `USB_DSB`, per-arch task stack sizes (soft-float ARMv6 frames are deeper), and an AArch64-only FPU enable — all behind `#if defined(__aarch64__)`. The Pi 3/400 build was re-synced from the Pi 1 work and confirmed on hardware, so the broadcast fix, the extra DMA barrier, and the robust hub scan landed on all targets from one source.

4. Application features (Pi side)

- **Networked clock / dashboard** — HTTP server with a web dashboard, HTTP Basic Auth, NTP time sync, DHCP with static-IP fallback.
 - **Scrolling marquee** — `msg <text>` scrolls across both a MAX7219 LED matrix and the HDMI view; driven locally or by pushed events relayed from the ESP32.
 - **QR rendering** — on-screen QR codes (`qr / qrsmall / qrfull`) via an embedded `qrcodegen`.
 - **HDMI instrument views** — logic-analyzer and oscilloscope frames streamed from the ESP32 are reassembled and painted on the framebuffer (`la_render / sc_render`).
 - **UART command console** (`krclock>` prompt) — a line shell: `set HH:MM:SS, time, msg, netmsg, qr/qrsmall, la/sc, run/stop/speed (LED sweep), leds <mask>, servo run|stop, brightness, and more.`
 - **Peripheral demos** — LED sweep, MAX7219, TM1637 7-segment, SPI, GPIO, servo, stepper, ADC.
-

5. The ESP32-S3 companion — console + instruments

The ESP32-S3 is not just a serial terminal; it is an active instrument front-end that offloads acquisition the bare-metal Pi has no peripherals for, and pushes results to the Pi over a simple UART line protocol.

5.1 Logic analyzer (BitScope BS05U → HDMI)

```
BS05U —FTDI(0403:6001)→ ESP32-S3 (native USB host)
|   BitScope VM: set regs · trigger · poll
· read N samples
|   decimate → up to 240 columns (1 byte =
8 channels)
```



```
UART @115200 — "la ..." lines → Pi —  
reassemble — paint 8 lanes
```

The ESP32 runs as a **USB host**, speaks the **BitScope VM** capture protocol to the BS05U (an FTDI serial device), packs samples into a compact wire frame, and streams it. At 240 columns / 4 fps the link runs at ~3.4 KB/s — about 30% of the 115200 line. The BVM command builder, sample packer, and framer are host-validated to round-trip with zero loss; only the USB-host bring-up needs a hardware pass.

Wire protocol (each line < 128 B, no acks on data so the stream doesn't flood the link):

```
la begin <ncols> <nch> <rate_hz>      start a frame  
la d <col_off> <hex>                 2 hex chars/col, bit i = channel  
i  
la end                                frame complete → repaint  
la off                                leave logic view
```

5.2 Oscilloscope (CHA/CHB → HDMI)

A parallel `sc ...` protocol carries up to two analog channels (8-bit samples, up to 480 columns) sampled on the ESP32's ADC, rendered by `sc_render` as waveform traces on HDMI.

5.3 Signal generator (self-test stimulus)

`siggen` drives A0..A3 (GPIO1.4) via LEDC: clean 50%-duty **square** waves to test the logic-analyzer lanes, and a PWM-approximated **sine** (the S3 has no DAC; a sine emerges after an external RC low-pass) to test the scope. This lets the whole acquisition + render path be validated with no external instrument, by looping a generator pin into a capture input.

5.4 Synthetic source

`la_sim` produces synthetic logic frames so the entire ESP32→Pi→HDMI path can be proven before the BS05U hardware is attached — the same discipline of validating the data path independently of the sensor.

5.5 Network relay + mobile control

- **WebSocket client** — connects to a backend (e.g. a Python WS server fronting MikroTik RouterOS netwatch). Each pushed event, plain text or JSON, is reformatted and forwarded to the Pi as `msg ...`, scrolling across the marquee. The Pi needs nothing new — it already turns any `msg` into a scroll.
- **RemoteXY over BLE** — a mobile GUI (timebase, trigger, view selection) using BLE so Wi-Fi stays free for **WiFiManager** runtime provisioning.

6. Toolchain and build

	Pi 1 (ARMv6)	Pi 3 / 400 (AArch64)	ESP32-S3
Compiler	arm-none-eabi-gcc	aarch64-*-gcc	Arduino / PlatformIO
Arch flags	-march=armv6z -	-mgeneral-regs -	esp32-s3-

	Pi 1 (ARMv6)	Pi 3 / 400 (AArch64)	ESP32-S3
	<code>mno-unaligned-access, soft-float</code>	<code>only, -nostdlib</code>	<code>devkitc-1</code>
libc	<code>newlib (-specs=nosys.specs)</code>	hand-written <code>kstring.c</code> shims	Arduino core
Float	<code>soft-float (no FPU dir)</code>	—	—
Source gather	<code>Makefile auto-globs drivers/*.c</code>	<code>explicit OBJD +=</code>	<code>lib_deps</code> (WiFiManager, RemoteXY, ArduinoJson, WebSockets)

The `-mno-unaligned-access` flag and `aligned_mem.c` are Pi 1-only: AArch64 does unaligned access in hardware, and adding `aligned_mem.c` there would collide with `kstring.c`'s own `memcpy/memmove`.

7. Engineering principles that held throughout

- **Hardware-confirm every step before the next.** Nothing was declared “done” from a successful compile or a QEMU run — only from real silicon, verified on the wire (hex dumps, packet captures) or on the screen.
- **Trace root cause to the bit before fixing.** The DHCP bug was chased through driver hex dumps, Mongoose bracketing logs, disassembly across every optimization level, and a boot-time `memcpy` self-test — until the exact rotation was explained — rather than patched by guess.
- **Validate the data path independently of the sensor.** Host-validated BVM framing, `la_sim` synthetic frames, and the `siggen` loopback each let a layer be proven before the hardware below it was trusted.
- **Isolate architecture differences, don't fork.** One driver, guarded at the few points the silicon actually differs, beats three diverging copies.
- **Keep third-party code stock.** Mongoose was never edited; the ARMv6 fix lives in the `libc` layer and a flag, so the network stack stays identical and upstream-updatable across all three boards.

8. Status and conclusion

Confirmed working on real hardware, all three Pi targets:

- USB / native Ethernet bring-up, link up.
- **DHCP lease completes on every board**, with static-IP fallback as a safety net.
- NTP time sync, HTTP dashboard with Basic Auth, WebSocket relay to the marquee.

- HDMI clock / marquee / QR, and the logic-analyzer / oscilloscope instrument views fed from the ESP32.
- Shared driver synced across Pi 1 / 3 / 400 from a single design.

The project set out to prove that a real, portable, from-scratch embedded stack — USB, networking, display, and instrumentation — could be brought up on bare metal across three Raspberry Pi generations and a microcontroller companion, without leaning on an OS or a vendor BSP. It does. The same application now runs on an ARMv6 Pi 1 with no MMU and no hardware unaligned access, on two AArch64 boards, and alongside an ESP32-S3 that turns the whole rig into a networked logic analyzer and oscilloscope — every layer traced to the register, and every layer confirmed on silicon.

The deepest lesson is the one the Pi 1 kept teaching: on hardware without the conveniences modern code assumes — cache coherence, unaligned access, an MMU — correctness is decided by details that are invisible in the source and only appear on the wire. The discipline of chasing them all the way down is what made the difference between “it compiles” and “it runs.”